

Tutorial 00 — Part 3

uv: Python Environments Without the Pain

init, add, sync, run — the whole lifecycle

Shuvam Banerji Seal

CH4114 — Molecular Simulation

14 August 2026

Repository: `CH4114_Molecular_Simulation_Tutorials`

The problem uv solves

The old way

```
python -m venv .venv
source .venv/bin/activate
pip install numpy mdtraj
pip freeze > requirements.txt
```

- pip freeze captures your *whole machine*, not the project.
- No Python version, no platform, no dev-vs-runtime split.
- Environments rot; “works on my machine”.

uv (by Astral)

One fast binary that manages interpreters, environments, dependencies, locking, and tools.
10–100× faster than pip.

What uv replaces

Old tool	Job	uv command
<code>python -m venv</code>	create env	<code>uv venv / uv sync</code>
<code>pip install</code>	add packages	<code>uv add</code>
<code>pip uninstall</code>	remove packages	<code>uv remove</code>
<code>requirements.txt</code>	pin versions	<code>uv.lock</code>
<code>pyenv</code>	manage Pythons	<code>uv python install</code>
<code>pip freeze</code>	snapshot env	<code>uv lock</code>
<code>npx-style tools</code>	run tools	<code>uvx</code>

- One mental model, one tool, one lockfile.

Installation on every OS

Linux

```
curl -LsSf https://astral.sh/  
uv/install.sh | sh  
# or: sudo apt install uv
```

macOS

```
curl -LsSf https://astral.sh/  
uv/install.sh | sh  
# or: brew install uv
```

Windows

```
powershell -ExecutionPolicy  
ByPass -c "irm https://  
astral.sh/uv/install.sh |  
iex"  
# or: winget install  
# --id=astral-sh.uv
```

Verify everywhere: `uv -version`

If uv: command not found: fix your PATH

Bash — ~/.bashrc

```
export PATH="$HOME/.local/bin:$PATH"
```

```
source ~/.bashrc
```

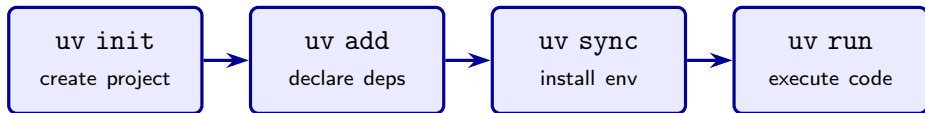
Zsh — ~/.zshrc

```
export PATH="$HOME/.local/bin:$PATH"
```

```
source ~/.zshrc
```

- Installer puts uv in ~/.local/bin (Homebrew: /opt/homebrew/bin).
- Verify: which uv and uv -version in a fresh terminal.

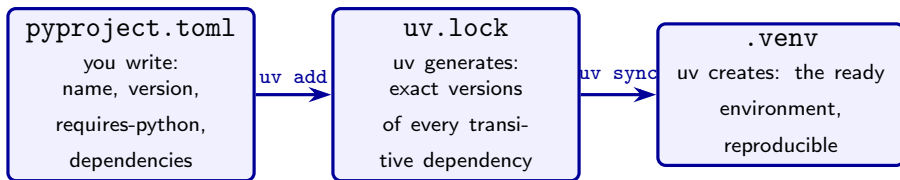
The main commands of this session



```
uv init demo && cd demo  uv add numpy mdtraj  uv sync  uv run python -c "import  
numpy"
```

- That is the entire lifecycle of a Python project in four commands.

The two-file contract



- Commit **both** files: `uv.lock` is the reproducibility contract.
- Anyone running `uv sync` gets byte-identical dependencies.

uv init: what you get

```
uv init my_project  
cd my_project
```

```
my_project/  
+-- .python-version  
+-- README.md  
+-- pyproject.toml  
+-- src/my_project/  
|   +-- __init__.py  
|   +-- main.py  
+-- .gitignore
```

- A ready-to-run project: metadata, package, gitignore, pinned Python.
- `uv run python src/my_project/main.py` prints the hello world.

uv sync and uv run

uv sync

`uv sync`

- creates `.venv/`
- resolves dependencies
- writes `uv.lock`
- installs the project (editable)

uv run — no activation needed

```
uv run python script.py
uv run pytest
uv run python -c "import numpy"
```

- `uv run` uses the project environment automatically.
- Manual activation still works: `source .venv/bin/activate`.

uv add and uv remove

Commands

```
uv add numpy
uv add "numpy>=2.0"
uv add mdtraj matplotlib
uv add --dev pytest
uv remove mdtraj
```

pyproject.toml after uv add

```
[project]
dependencies = [
    "mdtraj>=1.10",
    "numpy>=2.0",
]

[dependency-groups]
dev = [
    "pytest>=8.0",
]
```

- `uv add` edits `pyproject.toml` *and* `uv.lock`, then syncs — three pip steps in one.

Running tools: uv run and uvx

uv run — project tools

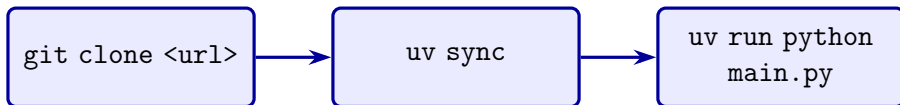
```
uv run python script.py
uv run pytest
uv run ruff check .
```

uvx — ephemeral tools

```
uvx ruff check .
uvx black --check .
uvx mamba --help
```

- uvx runs a tool in a throwaway environment — nothing pollutes your project.

Reproducing someone else's project



The result

Same lockfile → same environment → “works on my machine” is dead. Upgrades: `uv lock -upgrade` then `uv sync`.

Command	What it does
<code>uv init [name]</code>	scaffold a new project
<code>uv sync</code>	create env + install deps from lockfile
<code>uv add <pkg> / uv remove <pkg></code>	add / remove a dependency
<code>uv run <cmd></code>	run a command inside the project env
<code>uvx <tool></code>	run a tool in an ephemeral env
<code>uv lock / uv lock -upgrade</code>	(re)generate / upgrade the lockfile
<code>uv python install <ver></code>	install a Python interpreter
<code>uv python pin <ver></code>	pin the project interpreter
<code>uv venv</code>	create a bare virtualenv
<code>uv tree</code>	show the dependency tree
<code>uv cache clean</code>	clear the global cache

Worked example: a mini MD analysis project

Four commands, full stack

```
uv init md_analysis && cd md_analysis
uv add numpy mdtraj matplotlib
uv add --dev pytest
uv run python - <<'EOF'
import numpy as np, mdtraj as md
traj = md.load("traj.xtc", top="top.pdb")
rmsd = md.rmsd(traj, traj, 0)
print(f"frames: {traj.n_frames}, "
      f"mean RMSD: {np.mean(rmsd):.3f} nm")
EOF
```

- One environment, three packages, zero activation, fully locked.
- This is the workflow for every simulation project this semester.

Check yourself

- ❶ What is the difference between `pyproject.toml` and `uv.lock`? Which do you edit by hand?
- ❷ Why is `uv run` preferred over `source .venv/bin/activate`?
- ❸ A collaborator's repo has no `uv.lock` — which command generates one?
- ❹ What does `uv add -dev pytest` do differently from `uv add pytest`?
- ❺ Where does `uv` live, and what goes in `~/.bashrc` if `uv: command not found`?

Session complete

Git + GitHub → Project organization → uv environments. Now do the exercises!